Pointing OpenUSD at the Asset Administration Shell

A source-identifier hello-world — OpenUSD proposal PR 105, applied to AAS

An applied demonstration of Separation of Concerns for Identifiers (PR 105): a USD prim carries only a stable identifier; a runtime consumer resolves the live Asset Administration Shell data and does useful work with it — without copying that data into the scene.

The idea in one paragraph

An industrial asset shows up in a USD scene as a prim. The prim carries **one** thing: a stable, opaque `globalAssetId`. It does **not** carry a copy of the asset's data, and it does **not** carry the network endpoints where that data lives. At runtime, a consumer reads the `globalAssetId` off the prim, resolves it against an Asset Administration Shell (AAS) backend over the standard AAS REST APIs, and does real work with the result — here, it rolls up the asset's Product Carbon Footprint and flags where footprint data is missing. The USD scene stays a portable, durable description of *what the asset is*; the AAS stays the authoritative source of *the asset's data*; a single stable identifier connects the two.

Why this matters

Industrial digital twins — the kind that sit behind AI-factory reference designs like NVIDIA's publicly announced Omniverse DSX Blueprint — aggregate thousands of components from many vendors into one simulated, operated 3D scene. Each component has authoritative engineering data living in a system of record. There are two obvious ways to bring that data into the twin, and **both break the link back to the source**:

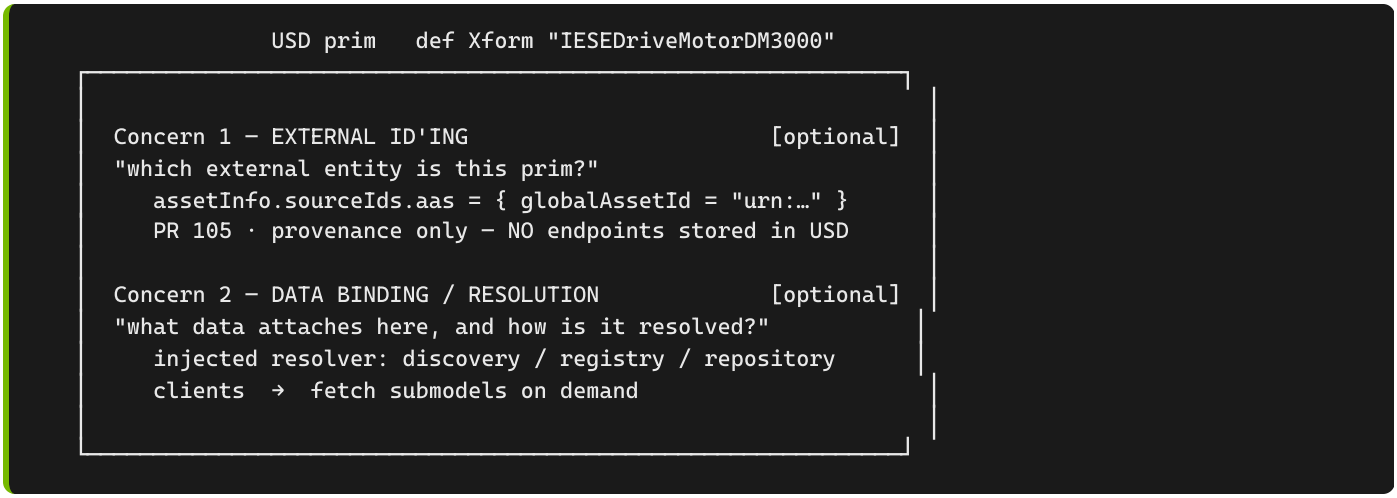
- **Copy the data into the scene.** It goes stale the moment the source changes. Worse, an aggregate built from copied data (a carbon-footprint roll-up, a compliance report) can't be trusted, because a *missing* value looks identical to a *zero*.
- **Bake the source endpoints into the scene.** The twin becomes brittle and non-portable: it can't move from one system, site, or lifecycle stage to the next without rewiring, and a single asset that resolves to several shells along a supply chain has nowhere to put them.

The **Asset Administration Shell** (IDTA specifications) is the open Industry 4.0 standard for industrial asset data. Each shell is addressable by a stable `globalAssetId` and carries submodels — Digital Nameplate, Technical Data, Handover Documentation, Product Carbon Footprint, and more. AAS gives us a standard, vendor-neutral place for the data of record to live. The question this hello-world answers is concrete: **how should a USD asset point at its authoritative AAS so the link survives, stays portable, and stays trustworthy?**

> This proof of concept assumes no specific product or vendor exposes its assets as AAS today. It builds the connection now — against open, standard tooling — so it pays off as adoption grows.

Two concerns, separated — the PR 105 idea

PR 105 — *Separation of Concerns for Identifiers* draws a line between two things that are easy to conflate:



The two are **independent and individually optional**. A prim can carry an identifier with no resolver attached; a resolver can be swapped without touching the prim. Keeping them separate is exactly what makes the scene portable: the identity is durable and travels with the asset, while resolution is a runtime concern supplied by whoever is consuming the scene, wherever they are.

Architecture

1 — The USD prim carries only the identifier. This is the entire asset on the USD side:

```
#usda 1.0
def Xform "World"
{
    def Xform "IESEDriveMotorDM3000" (
        assetInfo = {
            dictionary sourceIds = {
                dictionary aas = {
                    string globalAssetId = "urn:fraunhofer:iese:dte:asset:drivemotor-dm3000:serial-0042"
                }
            }
        }
    )
    {
    }
}
```

No endpoints. No copied submodels. Just the stable identity, in USD's existing `assetInfo` metadata — no new schema required, ships today.

2 — Resolution is injected at runtime. A single seam takes the stable identity plus a *context* — which backend(s) to resolve against for the current deployment — and walks the standard AAS API chain with failover:

```
def resolve(global_asset_id: str, context: ResolverContext) → ResolvedAsset:
    # Hop 1 — Discovery:  globalAssetId → [aasId, ...]
    # Hop 2 — Registry:   aasId → repository endpoint
    # Hop 3 — Repository: fetch the shell; submodel bodies fetched lazily
    ...
```

The endpoints live in the `context`, supplied at runtime — never in USD. The same `globalAssetId` resolves to different shells in different contexts, because the AAS network is decentralized. Real deployments may skip Discovery and start at Registry or Repository, with no marker for which — so the chain fails over. Submodel bodies are fetched lazily and on demand, so cost scales with what's *accessed*, not with scene size; the result is read-once and cacheable, keyed `(globalAssetId, context)`.

3 — The resolved data is used, not mirrored. The consumer does its work with the live AAS data and **does not write it back into the USD scene as attributes**. This is the binding-pointer posture: USD carries the identifier, the AAS holds the data, the consumer pulls it on demand. Mirroring AAS submodels into USD attributes would duplicate the source-of-record schema into the scene — exactly the staleness problem the architecture is built to avoid. (Whether to expose resolved data in-scene at all — via a runtime session layer or a procedural `SdfFileFormat` payload — is deliberately left out of scope.)

> **A note on the word "resolve."** The AOUSD Core Spec reserves *resolution* for USD **value resolution** (the strongest composed opinion at a path) and **asset resolution** (identifiers → locations via `Ar`). The `resolve()` here is **neither** — it is the AAS data-acquisition step (`globalAssetId` → live AAS data over HTTP). The term is kept for continuity with how practitioners describe the AAS lookup; a reader from a USD background should not read it as value or asset resolution.

What runs today

The hello-world runs end to end against **Eclipse BaSyx** (the open-source AAS stack, minimal example), which ships a public Fraunhofer-IESE `drivemotor-dm3000` shell:

1. Author `drivemotor.usda` — a prim carrying only the `globalAssetId`.
2. A runtime consumer reads that id, injects the BaSyx endpoint at runtime, and resolves the three-hop chain — reaching the live shell and its submodels.
3. The consumer parses the **CarbonFootprint** submodel (IDTA 02023, fields located by `semanticId` so it survives `idShort` variation between suppliers) and computes a cradle-to-gate Product Carbon Footprint:

EN 15804 stage	kg CO_2e / piece
A1 — raw materials	180
A2 — transport	25
A3 — manufacturing	80
Cradle-to-gate (A1–A3)	**285** ✓ complete
Use phase (B*), end-of-life (C*)	*not reported — flagged*

Nothing is written back into the USD scene. The roll-up is computed in the consumer and handed to the application.

Why gap-flagging is the point

A carbon roll-up you can put in a report has to distinguish "this component's footprint is zero" from "this component never reported a footprint." The consumer therefore treats coverage as a first-class output: it sums only the stages actually reported, and it names what's missing — a component with no PCF at all, an entry missing its CO_2e value, an incomplete cradle-to-gate, an unreported use phase. **A missing value is never silently coerced to zero.** That makes the aggregate reporting-grade and the gaps actionable — they tell you which supplier to chase. The roll-up logic is written to sum across many components, so it extends directly to a bill-of-materials.

Roadmap

- **Bill-of-materials roll-up (next).** Several components in one scene — each a prim with its own `globalAssetId`, at least one deliberately PCF-light — resolved together, with the carbon rolled up across the whole assembly and the gaps named. Same resolver, same roll-up logic, at assembly scale: where the value compounds for a real facility.
- **Change-driven refresh (later).** Because identity and data-fetch are separate concerns by design, change-driven refresh is incremental: when an AAS exposes a change/notification mechanism, the same resolved prims can subscribe and update in place. The proof of concept reads on demand (read-once) first, because AAS event/notification standardization is still maturing server-side. Pull-on-demand proves the binding; push-refresh follows when the source side is ready.

References

- **PR 105 — Separation of Concerns for Identifiers** · https://github.com/PixarAnimationStudios/OpenUSD-proposals/pull/105
- **AAS DPP ↔ OpenUSD proof of concept** (M. Wagner, SyncTwin) · https://github.com/asluk/OpenUSD-proposals/pull/2
- **AAS specifications** (IDTA — Metamodel Part 1, API Part 2, CarbonFootprint IDTA 02023) · https://industrialdigitaltwin.org/en/content-hub/aasspecifications
- **Eclipse BaSyx** (open-source AAS infrastructure) · https://github.com/eclipse-basyx/basyx-go-components
- **Omniverse DSX Blueprint** (domain motivation; public) · https://blogs.nvidia.com/blog/omniverse-dsx-blueprint/
- **EN 15804** — product-stage life-cycle modules (A1–A3 cradle-to-gate, B *use, C* end-of-life)